C++ 基础知识专题讲义

一维数组：从“连续内存”到下标访问

面向第一次系统学习数组的本科生·LaTeX 可视化版

这份讲义解决 6 个最基础的问题：

1. 数组究竟是什么，为什么它适合存一组同类数据？
2. 为什么下标从 0 开始，合法范围为什么是 $0 \sim N-1$？
3. 三种一维数组定义方式分别在做什么？
4. `sizeof(arr)` 为什么能得到整个数组的字节数？
5. “数组名就是地址”“数组名是常量”这两句话哪里不够严谨？
6. 这些知识以后怎样连接到指针、线性表、`std::array` 和 `std::vector`？

2026-09-05

目录

阅读说明：先建立“图像”，再记代码 **4**

1 原始笔记内容重编码与核对 **4**

1.1 原始笔记覆盖了哪些知识 4

1.2 将笔记示例规范化为可运行 C++ 4

1.3 原图中三条需要“升级严谨度”的表述 5

2 为什么需要数组：它解决的是“批量同类数据”问题 **6**

2.1 先看一个没有数组的程序 6

2.2 为什么“连续内存”特别重要 6

3 一维数组的声明与初始化：三种写法到底差在哪 **7**

3.1 第一种：只给长度，不给初值 7

3.2 第二种：给长度，也给初始化列表 7

3.3 第三种：省略长度，让编译器推断 7

3.4 数组长度不是普通运行时变量 8

4 下标访问：为什么从 0 开始，为什么最后一个是 $N-1$ **8**

4.1 下标不是“第几个”的自然语言编号 8

4.2 越界不是“自动报错” 8

5 遍历数组：把“逐格访问”变成循环 **8**

5.1 经典下标循环 9

5.2 范围 `for`：当你不关心下标时 9

6 `sizeof`：从“字节数”得到“元素个数” **10**

6.1 `sizeof(arr)` 问的是什么 10

6.2 把笔记中的代码写得更现代一点 10

7 数组名、首地址与指针：最容易被一句话讲歪的地方 **11**

7.1 “数组名就是首地址”为什么只适合当入门口令 11

7.2 为什么 `sizeof(arr)` 没变成指针大小 11

7.3 数组为什么不能整体赋值 11

8 把所有基础知识连成一个完整程序 **11**

9 高频易错点：数组题真正容易错在哪里 **12**

10 知识连接：为什么数组是数据结构和指针的门槛 **13**

10.1 前置知识：变量 + 循环 13

10.2 后续 1：指针 13
10.3 后续 2：数据结构 13
10.4 后续 3：std::array 与 std::vector 13

11 本专题知识地图与学习自检 14

12 正式基础练习（共 24 题） 15
12.1 单项选择题（10 题） 15
12.2 多项选择题（5 题） 16
12.3 填空题（5 题） 16
12.4 基础解答题（4 题） 17

13 答案速查 18

14 单项选择题详细解析 18

15 多项选择题详细解析 19

16 填空题详细解析 20

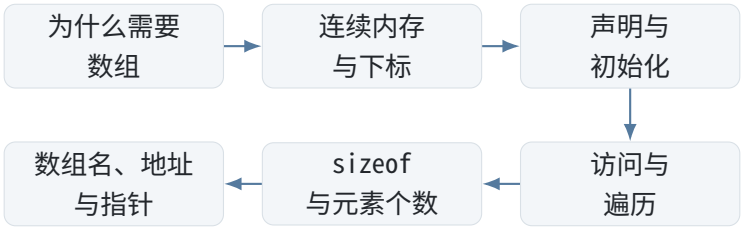
17 基础解答题详细解析 20

18 高频易错点总表与基础解题步骤 22

19 权威核验来源与术语依据 23

阅读说明：先建立“图像”，再记代码

上传笔记已经给出了数组的概念、三种定义方式、下标访问、for 遍历、sizeof 统计数组大小以及数组首地址等核心内容。本讲义不按“抄一遍笔记”的方式处理，而是把这些内容重新组织为一条连续的学习链：



学习策略

先把数组想成“一排编号固定、类型相同、地址连续的小格子”。随后所有语法都围绕这张图理解：声明是在“开多少格”；下标是在“选第几格”；遍历是在“按编号逐格访问”；sizeof 是在“问整排格子占多少字节”；数组到指针转换则是在“多数表达式里把整排数组变成指向第 0 格的指针值”。

1 原始笔记内容重编码与核对

1.1 原始笔记覆盖了哪些知识

根据三页手写材料，可以可靠识别出以下主线：

1. 数组的概念：一组相同类型的数据元素；
2. 数组的两个特点：**元素类型相同**、**内存连续**；
3. 一维数组的三种常用定义方式；
4. 用 arr[0], arr[1], ...访问元素；
5. 用 for 循环完成遍历；
6. 用 sizeof(arr) 统计整个数组所占字节数；
7. 用 sizeof(arr)/sizeof(arr[0]) 计算元素个数；
8. 输出数组表达式，观察其与首元素地址的关系。

1.2 将笔记示例规范化为可运行 C++

原笔记第一种写法中先声明了 int arr1[5];，随后手写赋值却写成了 arr[0]、arr[1]。为了保持变量名一致，规范版写为：

```cpp
#include <iostream>
using namespace std;

int main()
{
```

```cpp
    int arr1[5];
    arr1[0] = 1;
    arr1[1] = 2;

    cout << arr1[0] << '\n';

    int arr2[5] = {1, 2, 3, 4, 5};
    cout << arr2[3] << '\n';

    for (int i = 0; i < 5; ++i)
    {
        cout << "element " << i + 1 << ": " << arr2[i] << '\n';
    }

    int arr3[] = {1, 2, 3, 4, 5, 6};
    return 0;
}
```

原图纠错 1：变量名必须前后一致

如果声明的是 arr1，后续就必须写 arr1[0]，而不是 arr[0]。否则在没有另外声明 arr 的情况下，编译器会报告“标识符未声明”一类错误。这是**代码变量名笔误**，不是数组语法本身的问题。

1.3 原图中三条需要“升级严谨度”的表述

原图纠错 2：“数组名是常量”并不严谨

更准确的说法是：**数组对象本身不能用普通赋值运算符整体赋值**，例如两个内建数组 a 和 b 不能写 a = b;。这并不是因为“数组名是一个常量”。在大多数表达式中，数组表达式会发生**数组到指针转换**，得到指向首元素的指针值。

原图纠错 3：cout << arr 需要补充条件

若 arr 是 int 数组，那么在输出表达式中它通常转换为 int*，再以地址形式输出；但如果它是字符数组，cout << arr 可能按 C 风格字符串解释。因此初学阶段想明确“打印地址”，更稳妥的示例是 cout << static_cast<const void*>(arr);。

原图补充：省略数组长度时必须有足够信息推断长度

笔记中的 int arr3[] = \{1,2,3,4,5,6\}; 是合法的，因为初始化列表让编译器能够推断数组有 6 个元素。对一个普通局部数组，不能只写 int arr[]; 而不给初始化信息。

关于第 2 页红色手写批注的识别说明

该处红字逐字书写略难完全确认，但其与旁边第三种定义方式的关系是清楚的：它在提醒“省略方括号中的长度时，需要由初始化信息推断数组界限”。因此本讲义没有把不确定字迹冒充为逐字转录，而是按可核验的 C++ 规则写成上述补充说明。

2 为什么需要数组：它解决的是“批量同类数据”问题

2.1 先看一个没有数组的程序

假设要保存 5 次实验测得的温度。如果没有数组，可以写：

```
double t1 = 21.3;
double t2 = 21.8;
double t3 = 22.0;
double t4 = 21.6;
double t5 = 21.9;
```

这在 5 个数据时还能接受，但如果变成 100 个数据，变量命名、循环处理、统一求和都会迅速变得笨重。数组的核心价值就是：**用一个名字管理一批同类型对象，再用下标区分其中的元素。**

正式定义：内建数组（built-in array）

设元素类型为 T，元素个数为 N。一个“N 个 T 的数组”由**连续分配的、非空的 N 个 T 类型子对象**组成，这些对象称为数组元素，并编号为

$$0, 1, 2, \ldots, N-1.$$

其中编号 0 的元素是第一个元素。

2.2 为什么“连续内存”特别重要

把 `int arr[5] = \{10,20,30,40,50\};` 想成一排连续格子：



一句话直观理解

数组不是“5 个彼此无关的变量恰好名字相似”，而是一个整体对象，其中按顺序嵌着 5 个相同类型的元素。

如果某个平台上 `sizeof(int)` 为 4 字节，那么这 5 个元素通常会依次占据 5 个连续的 4 字节区域。于是，从第一个元素到第 i 个元素的地址位移具有规则性，这也是下标访问和指针运算能够高效工作的根本背景。

数组元素地址的直观关系（用于建立图像）

若首元素地址记为 A_0，每个元素占 $s = \mathrm{sizeof}(T)$ 个字节，则第 i 个元素的起始地址可直观写作

$$A_i = A_0 + i\,s.$$

初学阶段先把它理解为“每向后移动一个下标，就跨过一个元素的字节数”。后面学习指针时，会看到 C++ 已经把这种缩放自动封装在指针算术中。

3 一维数组的声明与初始化：三种写法到底差在哪

3.1 第一种：只给长度，不给初值

```
1 int arr1[5];
```

这句话完成两件事：元素类型是 int，元素个数是 5。对于函数体内这种自动存储期的基本类型数组，如果没有初始化器，元素不会自动变成 0，因此不要在赋值之前读取它们。

易错点：声明了空间，不等于已经有了可安全使用的数值

int arr1[5]; 只是创建 5 个 int 元素。为了让每个元素都有确定值，可以逐个赋值，或直接使用初始化列表。

3.2 第二种：给长度，也给初始化列表

```
1 int arr2[5] = {1, 2, 3, 4, 5};
```

这最适合初学阶段：长度写得明白，初值也写得明白。

如果初始化器数量少于数组长度，例如：

```
1 int a[5] = {7, 8};
```

那么结果是

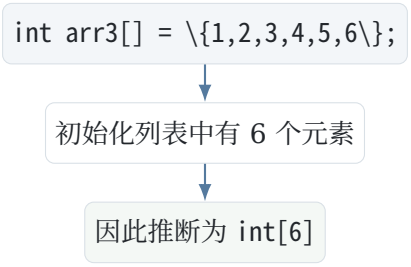
$$a = \{7, 8, 0, 0, 0\}.$$

剩余元素会被零初始化。

3.3 第三种：省略长度，让编译器推断

```
1 int arr3[] = {1, 2, 3, 4, 5, 6};
```

编译器从初始化列表中数出 6 个元素，因此 arr3 的类型可理解为“6 个 int 的数组”。



三种写法怎么选

写法	初学阶段的理解
int a[5];	长度确定，但基本类型元素尚未给定初值；随后必须在读取前赋值。
int a[5]=\{...\};	长度与初值都清楚，最直观。
int a[]=\{...\};	省略长度，利用初始化列表推断元素个数。

3.4 数组长度不是普通运行时变量

标准 C++ 的内建局部数组通常要求数组界限是编译期可确定的常量表达式。例如：

```
constexpr int N = 5;
int a[N] = {1, 2, 3, 4, 5};
```

如果元素数量需要在程序运行时动态决定，更适合使用 std::vector，这是后续容器知识。

4 下标访问：为什么从 0 开始，为什么最后一个是 $N-1$

4.1 下标不是“第几个”的自然语言编号

对于 5 个元素的数组 arr[5]，合法下标是

$$0, 1, 2, 3, 4.$$

因此：

- “第 1 个元素”对应 arr[0]；
- “第 2 个元素”对应 arr[1]；
- “第 5 个元素”对应 arr[4]。



合法下标范围

若数组长度为 N，则合法下标满足

$$\boxed{0 \le i < N}.$$

最大合法下标为

$$\boxed{N-1}.$$

这也解释了遍历循环为什么常写成 for (int i=0; i<N; ++i)。

4.2 越界不是“自动报错”

内建数组的 operator[] 不会帮你做运行时边界检查。访问 arr[5]（当长度为 5 时）已经越过合法区域，程序行为未定义。它可能暂时“看起来能运行”，也可能读到垃圾数据、破坏其他对象或直接崩溃。

最常见的边界错误

把循环条件写成 i <= 5 会让 i 最终取到 5，从而访问 arr[5]。长度为 5 的数组最后一个元素是 arr[4]，所以应写 i < 5。

5 遍历数组：把“逐格访问”变成循环

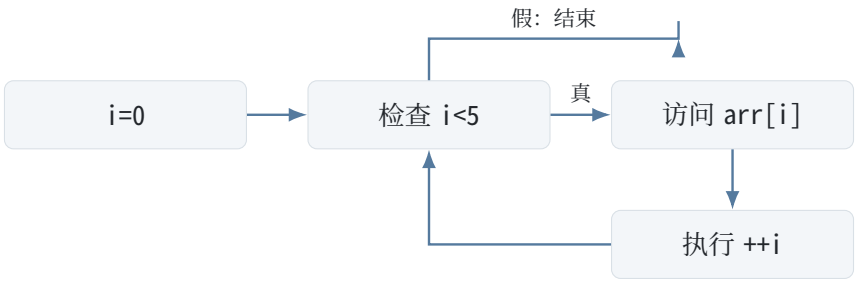
5.1 经典下标循环

```cpp
int arr[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; ++i)
{
    cout << arr[i] << '\n';
}
```

这个循环可以按四步理解：

1. i=0：从第 0 个元素开始；
2. i<5：只允许访问合法下标 0 到 4；
3. arr[i]：用当前下标选择一个元素；
4. ++i：移动到下一个下标。



5.2 范围 for：当你不关心下标时

C++11 起还可以写：

```cpp
for (int value : arr)
{
    cout << value << '\n';
}
```

它更像“依次取出数组中的每一个值”。但当前阶段仍建议把传统下标循环学扎实，因为后续数据结构、二维数组、索引算法都会频繁使用下标。

最基础应用：求 5 个数的和

```cpp
int a[5] = {1, 2, 3, 4, 5};
int sum = 0;

for (int i = 0; i < 5; ++i)
{
    sum += a[i];
}

cout << sum << '\n';   // 15
```

这里数组负责“保存一批同类数据”，循环负责“把同一种操作重复作用到每个元素”。这正是数

组最基础、最重要的使用模式。

6 sizeof：从“字节数”得到“元素个数”

6.1 sizeof(arr) 问的是什么

C++ 中 sizeof 返回对象或类型所占的字节数。对真正的数组对象使用 sizeof(arr) 时，得到的是**整个数组占用的总字节数**，而不是指针大小。

数组总字节数

若数组有 N 个元素，单个元素占 s 字节，则

$$\boxed{\text{sizeof}(arr) = N \times \text{sizeof}(arr[0])}.$$

所以可以反过来得到

$$\boxed{N = \frac{\text{sizeof}(arr)}{\text{sizeof}(arr[0])}}.$$



例：常见平台上 sizeof(int)==4 时

对 int arr[5] = \{10,20,30,40,50\};:

$$\text{sizeof}(arr) = 5 \times 4 = 20 \text{ 字节}, \qquad \frac{20}{4} = 5.$$

注意：sizeof(int) 的具体数值由实现决定，不能把“int 永远 4 字节”当成 C++ 标准保证。

6.2 把笔记中的代码写得更现代一点

笔记中的写法完全可以用于认识原理：

```
int arr[] = {1, 2, 3, 4, 5};
cout << sizeof(arr) << '\n';
cout << sizeof(arr) / sizeof(arr[0]) << '\n';
```

C++17 以后，如果引入相应头文件，也可以用 std::size(arr) 直接得到元素个数：

```
#include <iterator>
cout << std::size(arr) << '\n';
```

重要限制：进入函数参数后，sizeof(a)/sizeof(a[0]) 可能失效

函数参数写成 void f(int a[]) 时，该参数实际按指针类型处理；在函数体里，sizeof(a) 得到的是指针大小，而不是调用者数组的总大小。因此“数组长度公式”只在 a 仍然是实际数组对象

的作用域内可靠。

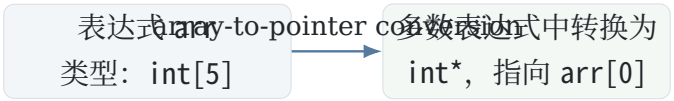
7 数组名、首地址与指针：最容易被一句话讲歪的地方

7.1 “数组名就是首地址”为什么只适合当入门口令

对 int arr[5];，标识符 arr 在语义上首先**表示这个数组对象**。但在多数普通表达式里，数组表达式会发生“数组到指针转换”，得到一个指向第一个元素的指针：

$$arr \quad \longrightarrow \quad \&arr[0].$$

所以很多时候你会观察到 arr 和 \&arr[0] 打印出的数值地址相同。



7.2 为什么 sizeof(arr) 没变成指针大小

因为 sizeof 是一个关键例外：对其操作数不会进行数组到指针转换。因此 sizeof(arr) 仍然看见“整个数组”，所以能得到数组总字节数。

同样地，\&arr 取得的是“整个数组对象的地址”，其类型是“指向 5 个 int 的数组的指针”，即 int (*)[5]，这与 int* 不是同一个类型。

三种看起来相近的表达式

表达式	初学阶段应如何理解
arr	多数表达式中转换为指向首元素的 int*。
&arr[0]	明确取得第 0 个元素的地址，类型 int*。
&arr	取得整个数组对象的地址，类型 int (*)[5]。

7.3 数组为什么不能整体赋值

下面代码不合法：

```
int a[3] = {1, 2, 3};
int b[3] = {4, 5, 6};
// a = b;   // error: built-in arrays are not assignable
```

这说明内建数组与普通标量对象不同：它没有提供普通的整体赋值语义。若需要“像一个值一样整体复制”的固定长度数组，可以学习 std::array；若需要可变长度容器，则常用 std::vector。

8 把所有基础知识连成一个完整程序

下面这段代码覆盖：初始化、元素访问、遍历、元素个数、总字节数和首元素地址。

```
#include <iostream>
#include <iterator>
```

```cpp
using namespace std;

int main()
{
    int arr[] = {10, 20, 30, 40, 50};

    cout << "arr[0] = " << arr[0] << '\n';
    cout << "arr[3] = " << arr[3] << '\n';

    cout << "elements = " << std::size(arr) << '\n';
    cout << "bytes = " << sizeof(arr) << '\n';

    cout << "values: ";
    for (int i = 0; i < static_cast<int>(std::size(arr)); ++i)
    {
        cout << arr[i] << ' ';
    }
    cout << '\n';

    cout << "first element address = "
         << static_cast<const void*>(arr) << '\n';

    return 0;
}
```

> **读程序时不要逐行死记**
>
> 把它翻译成自然语言：先开 5 个连续的 int 格子并放入数据；用下标选择格子；用 std::size 数格子；用 sizeof 量整排格子占多少字节；用循环从 0 号格走到最后一格；最后观察第 0 个格子的地址。

9 高频易错点：数组题真正容易错在哪里

错误类型	典型错误	正确理解
变量名不一致	声明 arr1，后面却访问 arr	标识符必须一致；这是编译错误。
越界	for(i=0;i<=5;++i) 访问长度 5 数组	合法下标是 0–4，循环应写 i<5。
把自然序号当下标	“第 1 个”写成 arr[1]	第 1 个元素是 arr[0]。
未初始化就读取	int a[5]; cout<<a[0];	基本类型局部数组未给初值时不要直接读取。
误记数组名	“数组名是常量，所以不能赋值”	不能整体赋值的原因是内建数组本身不可赋值；数组名不是简单意义上的常量。
误用长度公式	在函数参数 int a[] 内计算 sizeof(a)/sizeof(a[0])	此处 a 实际按指针参数处理，数组总大小信息已不在参数类型中。

错误类型	典型错误	正确理解
把 sizeof 当元素个数	sizeof(a) 直接当长度	它返回字节数；元素个数要除以单个元素字节数或用 std::size。

调试顺序

遇到数组错误时，优先按这个顺序检查：**变量名** → **数组长度** → **下标范围** → **是否先初始化后读取** → **循环边界** → **是否在函数参数里误用 sizeof**。

10 知识连接：为什么数组是数据结构和指针的门槛

10.1 前置知识：变量 + 循环

数组本身并不难，它主要把你已经学过的两个工具连接起来：变量负责保存数据，循环负责重复操作。数组让“循环的第 i 次”自然对应“数组的第 i 个下标”。

10.2 后续 1：指针

由于数组元素连续，而且数组在多数表达式中会转换为指向首元素的指针，所以学习数组后很快会进入：

$$arr[i] \quad \leftrightarrow \quad *(arr+i).$$

这不是要求你现在立刻使用指针写数组程序，而是解释为什么“连续内存 + 下标”会成为指针章节的前置知识。

10.3 后续 2：数据结构

数据结构课程中的顺序表，本质思想就是把元素放在**逻辑顺序与物理存储顺序相一致**的连续区域中。你现在学到的“连续内存、下标、遍历、长度”会直接变成顺序表、栈、队列、矩阵等内容的基础语言。

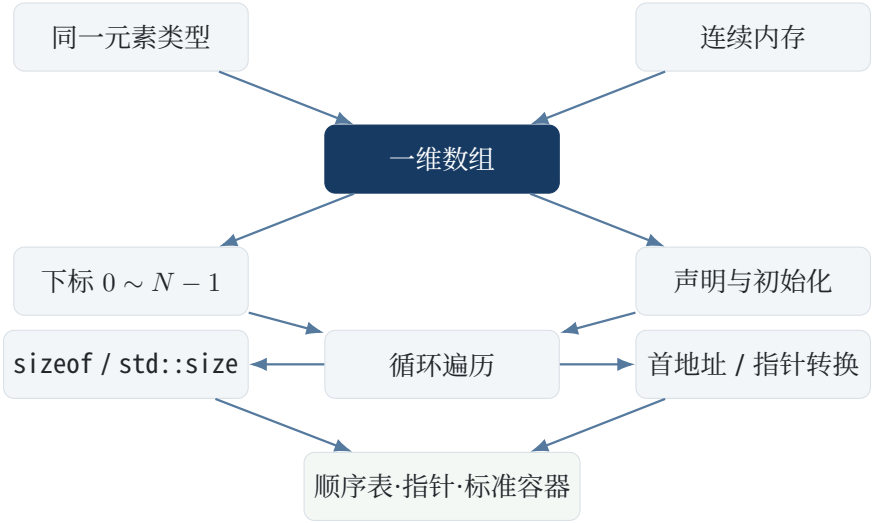
10.4 后续 3：std::array 与 std::vector

	内建数组	std::array	std::vector
长度	编译期固定	编译期固定	运行时可变
整体赋值	不支持	支持	支持
size()	无成员函数	有	有
连续存储	是	是	是（元素连续）
适合当前阶段	必学基础	紧接着可学	动态序列的核心容器

拓展联想：当前阶段不要求掌握

现代 C++ 实际工程中通常更偏好能表达长度、拥有成员函数并能与标准库算法自然协作的容器类型。但内建数组仍然非常重要，因为它让你真正理解“固定长度、连续存储、下标、地址、退化”这些底层概念。

11 本专题知识地图与学习自检



学完这里，你应该能够回答

1. 数组为什么要求元素类型相同，为什么强调连续内存？
2. 长度为 N 的数组合法下标为什么是 0 到 $N-1$？
3. 三种常用一维数组定义方式分别是什么？
4. `int a[5]=\{1,2\};` 的后 3 个元素是什么？
5. 为什么传统遍历循环常写 `i < N` 而不是 `i <= N`？
6. `sizeof(a)` 与 `sizeof(a[0])` 分别表示什么？
7. 为什么“数组名就是首地址”和“数组名是常量”都只是过度简化的说法？
8. 什么时候 `sizeof(a)/sizeof(a[0])` 会失效？

12 正式基础练习（共 24 题）

12.1 单项选择题（10 题）

1. 关于 C++ 内建数组，下列说法正确的是（ ）。
A. 同一个数组中可以随意混放 int 和 double 元素
B. 数组元素具有相同的元素类型
C. 数组下标一定从 1 开始
D. 数组元素在内存中一定彼此分散

2. 已知 int a[5];，下列下标中合法的是（ ）。
A. a[-1] B. a[0] C. a[5] D. a[6]

3. 语句 int a[] = \{3,5,7,9\}; 定义的数组元素个数是（ ）。
A. 3 B. 4 C. 5 D. 无法确定

4. 已知 int a[5]=\{10,20,30,40,50\};，表达式 a[2] 的值是（ ）。
A. 10 B. 20 C. 30 D. 40

5. 对真正的数组对象 int a[5];，用于计算元素个数的经典表达式是（ ）。
A. sizeof(a)
B. sizeof(a[0])
C. sizeof(a)/sizeof(a[0])
D. sizeof(a[0])/sizeof(a)

6. 执行 int a[5]=\{1,2\}; 后，a[4] 的值是（ ）。
A. 0 B. 2 C. 未定义且一定不能为 0 D. 5

7. 遍历 int a[5]; 的正确循环条件最典型的是（ ）。
A. i <= 5 B. i < 5 C. i > 5 D. i == 5

8. 下列对“数组名是常量”的评价最准确的是（ ）。
A. 完全严格，因为数组名的类型就是 const int
B. 不够严谨；更准确地说，内建数组本身不能整体赋值，而且多数表达式中数组会转换为指向首元素的指针
C. 完全错误，因为数组可以写 a=b
D. 数组名一定是整数常量

9. 对 int a[3]=\{1,2,3\};，在多数普通表达式中 a 会转换为（ ）。
A. 指向首元素的 int*
B. 元素个数 3
C. 最后一个元素的值
D. double*

10. 对长度为 5 的数组访问 a[5]，正确说法是（ ）。
A. 一定返回 0

B. 一定自动扩容
C. 已越界，属于未定义行为
D. 等价于 a[4]

12.2 多项选择题（5 题）

1. 关于数组的基本性质，正确的有（　）。
A. 元素编号从 0 开始
B. 元素连续存储
C. 同一数组的元素类型相同
D. 长度为 N 时最大合法下标为 N

2. 下列声明在标准语义下可以正确表示一个已定义的普通局部 int 数组的有（　）。
A. int a[3] = \{1,2,3\};
B. int b[] = \{1,2,3\};
C. int c[3];
D. int d[];

3. 已知 int a[4]=\{1,2,3,4\};，下列说法正确的有（　）。
A. sizeof(a) 是整个数组的总字节数
B. sizeof(a[0]) 是一个元素的字节数
C. sizeof(a)/sizeof(a[0]) 的结果为 4
D. sizeof(a) 的结果一定是 4

4. 下列循环可以在不越界的前提下访问长度为 5 的数组所有元素的有（　）。
A. for(int i=0;i<5;++i)
B. for(int i=4;i>=0;--i)
C. for(int i=1;i<=5;++i) cout<<a[i];
D. 范围 for(int x : a)

5. 关于数组与指针，下列说法正确的有（　）。
A. 数组表达式在多数普通表达式中可转换为指向首元素的指针
B. sizeof(a) 对实际数组对象不会先把 a 转换成指针
C. 函数参数写成 int a[] 时，在函数体内按指针参数处理
D. 两个内建数组可以直接用 a=b 整体赋值

12.3 填空题（5 题）

1. 长度为 N 的数组，最后一个元素的下标是 _____。

2. 数组 int a[5]; 的第一个元素写作 _____。

3. 对实际数组对象 a，经典元素个数公式是 _____。

4. int a[] = \{2,4,6\}; 中，编译器推断数组长度为 _____。

5. 从下标 0 遍历长度为 5 的数组，循环条件通常写成 i _____ 5。

12.4 基础解答题（4 题）

1. 定义一个包含 10,20,30,40,50 的 int 数组，并用下标循环从前到后输出全部元素。要求说明为什么循环条件使用 i<5。

2. 已知 int a[5]=\{1,2,3,4,5\};，编写程序片段求所有元素之和，并说明 sum += a[i] 每次循环做了什么。

3. 找出并改正下面代码中的两个错误：

```
int arr1[5] = {1, 2, 3, 4, 5};
for (int i = 0; i <= 5; ++i)
{
    cout << arr[i] << '\n';
}
```

4. 已知：

```
int a[5] = {1, 2, 3, 4, 5};
```

解释 sizeof(a)、sizeof(a[0])、sizeof(a)/sizeof(a[0]) 三个表达式分别表示什么；再说明为什么把 a 作为 void f(int a[]) 的参数后，不能在 f 内继续用同样公式可靠得到调用者数组长度。

13 答案速查

单项选择题

1.B 2.B 3.B 4.C 5.C 6.A 7.B 8.B 9.A 10.C

多项选择题

1.ABC 2.ABC 3.ABC 4.ABD 5.ABC

填空题

1. $N-1$； 2. a[0]； 3. sizeof(a)/sizeof(a[0])； 4. 3； 5. <。

解答题核心结论

1. 下标 0–4 遍历； 2. 和为 15； 3. arr 应为 arr1，i<=5 应改为 i<5； 4. 分别是总字节数、单元素字节数、元素个数；函数数组参数按指针处理。

14 单项选择题详细解析

第 1 题

正确答案：B。 数组的核心类型特征是所有元素具有同一元素类型；同时标准数组元素是连续分配的。A 把同一数组描述成任意混合类型，错误；C 把下标起点写成 1，错误；D 与连续存储相反。**一句话：同类型 + 连续 + 0 起始编号。**

第 2 题

正确答案：B。 长度为 5 的合法下标满足 $0 \le i < 5$，即 0、1、2、3、4。a[0] 合法；负下标和 5、6 都越界。常见错误是把“5 个元素”误解成“最后一个下标是 5”。

第 3 题

正确答案：B。 省略显式长度后，编译器根据初始化列表 \{3,5,7,9\} 中的 4 个元素推断数组长度为 4。它不是“长度未知的可变数组”。

第 4 题

正确答案：C。 a[0]=10，a[1]=20，a[2]=30。下标 2 对应自然语言中的第 3 个元素。A、B、D 分别对应其他下标。

第 5 题

正确答案：C。 sizeof(a) 是整个数组字节数；sizeof(a[0]) 是单个元素字节数。两者相除，字节单位抵消，得到元素个数。A 只是总字节数，B 只是单元素字节数，D 比值方向反了。

第 6 题

正确答案：A。 初始化列表只显式给了前两个元素 1 和 2，剩余元素进行零初始化。因此最后三个元素都是 0。它与完全不写初始化器的 `int a[5];` 不同。

第 7 题

正确答案：B。 从 0 开始遍历 5 个元素，需要依次访问 0、1、2、3、4，因此继续循环的条件是 `i<5`。若写 `i<=5`，最后会访问 `a[5]` 越界。

第 8 题

正确答案：B。 “数组名是常量”只是某些入门材料用来阻止初学者写 `a=b` 的口令，并非严格类型描述。更准确的是：内建数组对象不可通过普通赋值运算符整体赋值；多数普通表达式中数组表达式还会转换为指向首元素的指针。

第 9 题

正确答案：A。 在多数普通表达式中，数组 `a` 发生数组到指针转换，结果指向 `a[0]`。这也是为什么很多场景下直接输出整型数组表达式会观察到一个地址样式的值。

第 10 题

正确答案：C。 长度为 5 的数组只包含下标 0–4。`a[5]` 越界，内建数组不会自动扩容或自动返回 0；越界访问导致未定义行为。

15 多项选择题详细解析

第 1 题

正确答案：ABC。 **A 正确：**元素编号从 0 开始。**B 正确：**元素连续分配。**C 正确：**同一数组元素类型一致。**D 错误：**最大合法下标是 $N-1$，不是 N。

第 2 题

正确答案：ABC。 **A 正确：**长度与初始化器都明确。**B 正确：**可由 3 个初始化值推断长度。**C 正确：**可以只声明长度；只是基本类型局部元素未初始化，读取前应赋值。**D 错误：**普通局部数组既省略长度又没有初始化器，缺少确定数组界限的信息。

第 3 题

正确答案：ABC。 **A 正确：**`sizeof(a)` 对实际数组对象得到总字节数。**B 正确：**`sizeof(a[0])` 得到一个 `int` 元素的字节数。**C 正确：**总字节数除以单元素字节数得到 4。**D 错误：**`sizeof(a)` 的单位是字节，结果取决于 `sizeof(int)`；并不等于元素个数 4。

第 4 题

正确答案：ABD。 **A 正确：**访问 0–4。**B 正确：**倒序访问 4、3、2、1、0。**C 错误：**访问 1–5，会漏掉 `a[0]` 并越界访问 `a[5]`。**D 正确：**范围 `for` 可依次访问每个元素。

第 5 题

正确答案：ABC。 **A 正确：**这是数组到指针转换的核心。**B 正确：**sizeof 的数组操作数不进行该转换。**C 正确：**函数参数中的数组写法会调整为指针参数。**D 错误：**两个内建数组不能直接整体赋值。

16 填空题详细解析

第 1 题

标准答案：$N-1$。 因为第一个元素从 0 编号，长度为 N 时编号依次为 $0,1,\ldots,N-1$。最常见误答是 N。

第 2 题

标准答案：a[0]。 数组下标从 0 开始，因此第一个元素不是 a[1]。

第 3 题

标准答案：sizeof(a)/sizeof(a[0])。 分子是整个数组字节数，分母是一个元素字节数，商为元素数量。前提是这里的 a 仍然是真正的数组对象。

第 4 题

标准答案：3。 初始化列表有 3 个值，编译器据此把 a 推断为 3 个 int 的数组。

第 5 题

标准答案：<。 因为 i 从 0 开始，允许 0–4，达到 5 时应停止，因此条件写 i<5。

17 基础解答题详细解析

第 1 题：定义并遍历 5 个元素

题意分析：既要定义数组，又要用下标循环完整访问一次。
已知：元素是 10、20、30、40、50。
求：数组定义与遍历代码。

```
int a[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; ++i)
{
    cout << a[i] << '\n';
}
```

第一步：数组长度为 5，所以合法下标是 0–4。
第二步：i=0 让第一次循环访问 a[0]。
第三步：i<5 保证最后一次合法访问是 i=4；当 i 变为 5 时循环立即结束。
检查：不会遗漏第 0 个元素，也不会访问 a[5]。
同类题方法：只要把 5 换成实际数组长度 N，循环模板就是 for(int i=0;i<N;++i)。

第 2 题：求数组元素之和

知识定位：“数组保存数据 + 循环逐个访问 + 累加变量保存阶段结果”。

```cpp
int a[5] = {1, 2, 3, 4, 5};
int sum = 0;

for (int i = 0; i < 5; ++i)
{
    sum += a[i];
}

cout << sum << '\n';
```

推理过程：

$$
\begin{aligned}
&\text{初始} sum = 0,\\
&i = 0: \quad sum = 0 + 1 = 1,\\
&i = 1: \quad sum = 1 + 2 = 3,\\
&i = 2: \quad sum = 3 + 3 = 6,\\
&i = 3: \quad sum = 6 + 4 = 10,\\
&i = 4: \quad sum = 10 + 5 = 15.
\end{aligned}
$$

结果：15。 `sum += a[i]` 等价于 `sum = sum + a[i]`，即把当前元素加入已经累积的结果。

第 3 题：诊断两个错误

原代码：

```cpp
int arr1[5] = {1, 2, 3, 4, 5};
for (int i = 0; i <= 5; ++i)
{
    cout << arr[i] << '\n';
}
```

错误 1：变量名不一致。声明的是 `arr1`，访问时却写成 `arr`。应统一为 `arr1[i]`。
错误 2：循环边界越界。 `i<=5` 会让 `i` 取到 5，进而访问第 6 个位置 `arr1[5]`；但长度 5 数组最大合法下标是 4。

规范代码：

```cpp
int arr1[5] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5; ++i)
{
    cout << arr1[i] << '\n';
}
```

同类题以后怎么做：看到数组循环，先在草稿上写出合法下标集合，再检查循环是否恰好覆盖这个集合。

第 4 题：解释 sizeof 与函数参数中的变化

第一步：解释三个表达式。

- `sizeof(a)`：整个 5 元素数组所占的总字节数；
- `sizeof(a[0])`：第 0 个元素（一个 `int`）所占字节数；
- `sizeof(a)/sizeof(a[0])`：总字节数除以单元素字节数，得到元素个数 5。

第二步：说明为什么函数参数里不同。

当函数写为：

```
void f(int a[])
{
    // here, a is treated as a pointer parameter
}
```

参数类型会调整为指针形式。此时 `sizeof(a)` 看见的是一个指针，不再是调用者那个完整数组对象，于是原来的“总字节数/单元素字节数”逻辑被破坏。

正确做法：初学阶段可以把长度作为另一个参数显式传进函数：

```
void f(int a[], int n)
{
    for (int i = 0; i < n; ++i)
    {
        cout << a[i] << '\n';
    }
}
```

后续还会学到用数组引用、`std::array`、`std::span` 等方式保留或表达长度信息。

18 高频易错点总表与基础解题步骤

数组基础题 6 步检查法

1. 看元素类型：是否同一类型；
2. 看数组长度：记为 N；
3. 写合法下标：$0 \sim N-1$；
4. 看初始化：是否每个需要读取的元素都有确定值；
5. 看循环边界：是否满足 `0 <= i < N`；
6. 如果出现 `sizeof` 或函数参数，再判断当前变量究竟还是数组，还是已经按指针处理。

容易混淆	正确区分	一句话记忆
第 k 个元素 vs 下标 k	第 k 个元素的下标是 $k-1$	自然序号从 1 开始，下标从 0 开始。
数组大小 vs 元素个数	`sizeof` 给字节数，`std::size` 给元素数	字节数不是元素数。
数组 vs 指针	数组是一个包含多个元素的对象；多数表达式中可转换为首元素指针	“会转换”不等于“本来就是”。
未初始化 vs 部分初始化	`int a[5];` 与 `int a[5]=\{1,2\};` 行为不同	有初始化列表时剩余元素会零初始化。

19 权威核验来源与术语依据

本讲义的核心定义、数组连续性、下标编号、数组界限省略、`sizeof` 行为以及数组到指针转换等内容，参考并交叉核验了以下资料（检索日期：2026-09-05）：

1. C++ Working Draft: [dcl.array] Arrays.
2. C++ Working Draft: [expr.sizeof] Sizeof.
3. cppreference: Array declaration.
4. cppreference: Implicit conversions - Array-to-pointer conversion.
5. cppreference: std::size.
6. Microsoft Learn: Arrays (C++).

资料使用说明

C++ 工作草案用于核对标准术语与语言规则；cppreference 用于交叉检查常见语法和标准库接口；Microsoft Learn 用于补充面向学习者的数组初始化与函数参数说明。对“数组名是常量”这类课堂口令，本讲义保留其教学动机，但使用更严格的语言规则重新表述。

——本专题结束：下一步建议学习“数组与指针的关系”以及“二维数组”——